Tech Stack

Last verified: **not verified.** Nothing in this file has been installed, built, or run. Every command below is a proposed interface from the Tech Design, not an observed result. Update this line only after you actually run the commands and record what happened.

Stack

Area	Choice	Notes
Frontend	Jinja2 server-rendered HTML, local CSS, minimal JavaScript polling (every 2 s while work runs)	One language for most logic; no frontend build step, so there is no browser compile command
Backend	Python 3.12 + FastAPI + Uvicorn	Expensive work runs in separate worker processes, never on the HTTP request thread
Database	SQLite via Python's built-in interface, explicit SQL migrations, plus immutable local artifact files	No database service; UUID text IDs, UTC ISO timestamps, foreign keys, schema-version table, short transactions
Auth	Local session token + `HttpOnly SameSite=Strict` cookie, CSRF on state-changing requests, loopback-only (`127.0.0.1`)	No accounts, OAuth, or remote access. Bootstrap a one-time token from the launcher, strip it from the address bar, and reject reuse
Styling	Local CSS	Neutral typography, high contrast, text/icons as well as colour for status; no CSS framework chosen
Deployment	One Ubuntu 24.04 LTS amd64 computer running the GUI; disposable Ubuntu x86-64 VM for all untrusted work	No hosting, cloud database, payments, email, analytics, or account system is required
Validation	Pydantic request/response models; pytest; Ruff; mypy	Local validation is never delegated to the provider
Workers	Ghidra/JDK, Capstone, LIEF, LLVM/Clang, binutils/GDB, angr/Claripy/Z3, one reintegration backend	Joint compatibility is unproven; freeze a tested set instead of newest-of-each

Baseline being proposed: Ubuntu 24.04 LTS amd64 host and guest, Python 3.12, Chromium/Chrome for the primary demonstration and Firefox for a smoke test. Planning allowance is 4 guest vCPUs, 8 GiB guest RAM, 30 GiB guest disk, on a host with 16 GiB RAM and 50 GiB free disk — estimates, not measured requirements. Record the actual host architecture, Ubuntu release, RAM, free disk, and virtualization support before planning around them; "Ubuntu" alone does not confirm an x86-64 CPU.

Commands

All of the following are **unverified proposals** that assume a project root with `pyproject.toml`. They are POSIX paths for the Ubuntu target. This working copy is currently on Windows, so run them inside the Ubuntu environment rather than translating them, and replace them with what actually passes.

- Setup: `python3 -m venv .venv && .venv/bin/python -m pip install -e '.[dev]'` — unverified; no `pyproject.toml` exists yet, so this cannot succeed today.
- Dev: `.venv/bin/python -m uvicorn app.main:app --host 127.0.0.1 --port 8000` — unverified.
- Test: `.venv/bin/python -m pytest` — unverified; no tests exist yet.
- Typecheck: `.venv/bin/python -m mypy app` — unverified.
- Lint/format: `.venv/bin/python -m ruff check .` — unverified.
- Build: none. There is no packaging or distribution command yet; the Tech Design leaves this empty deliberately. Do not invent one.
- Browser/device check: manual flow in Chromium/Chrome, plus Firefox smoke test — planned, not performed. Browser automation with Playwright Python is planned for the critical journey once the app exists.
- Dependency audit: `pip-audit` (or equivalent) — proposed, not yet chosen or run.

Behaviour the app must have on startup: the API starts with execution **disabled** until the isolation self-check passes. A GUI-only development mode using stored or synthetic results is allowed, must be clearly labelled, and must never emit accepted-patch evidence.

Configuration names

Proposed by the Tech Design; not created yet.

Name	Purpose
`DEEPSEEK_API_KEY`	Provider credential — trusted provider process only
`DEEPSEEK_BASE_URL`	Configurable provider endpoint
`DEEPSEEK_MODEL`	Pinned model/version after smoke test
`BINARYREPAIR_DATA_DIR`	Root for job artifacts and the SQLite database
`BINARYREPAIR_EXECUTION_ENABLED`	Gate for guest execution; defaults off
`BINARYREPAIR_VM_PROFILE`	Which guest/snapshot profile to use
`BINARYREPAIR_JOB_COST_LIMIT_USD`	Per-job spending cap
`BINARYREPAIR_CANDIDATE_LIMIT`	Candidate attempts allowed per route

The coding agent's own configuration is separate from these and is still TBD.

Project layout

To be created during implementation; the Tech Design specifies these responsibilities rather than delivered files.

Path	Responsibility
`app/main.py`, `app/api/`	FastAPI entry, local-session checks, typed routes
`app/templates/`, `app/static/`	Setup/review, comparison, evidence views; CSS; status polling
`app/domain/`	Contracts, candidate schemas, phases/outcomes, acceptance rules
`app/services/`	Job controller, provider adapter, artifact/export services
`app/db/`, `migrations/`	Parameterized SQLite access and schema migrations
`workers/analysis/`	Headless Ghidra and ELF metadata extraction
`workers/route_a/`, `workers/route_b/`	Route adapters and the Route B round-trip gate
`workers/verification/`	Structural, concrete, and symbolic checks
`workers/broker/`	Narrow VM task protocol and resource enforcement
`tests/unit/`, `tests/integration/`, `tests/e2e/`, `tests/evals/`	Acceptance semantics, worker integration, GUI, AI boundary scenarios
`fixtures/public/`	Public binary inputs and provenance, with no evaluator source
`var/jobs/`	Runtime artifacts — excluded from source control
`docs/`	PRD, research, technical design, decisions, replay guide
`pyproject.toml`, `requirements.lock`	Project metadata and the tested dependency lock

The evaluator source/build workspace lives **outside** this checkout. A folder merely hidden from the GUI does not count as separation.

Planned API surface

Proposed contracts, useful for checking that an implementation matches the design. Not implemented.

Method and path	Contract
`POST /api/jobs`	Multipart ELF, typed test manifest, defect details; returns job ID and original hash
`POST /api/jobs/{id}/analysis`	Queue screening and binary-derived analysis; returns 202 with a run/task identifier

Method and path	Contract
`GET /api/jobs/{id}`	Summary, current contract, phases, capabilities, outcomes
`PUT /api/jobs/{id}/contract`	Validated contract body plus expected revision; returns new revision/hash
`POST /api/jobs/{id}/approve`	Exact expected input/contract/manifest/scope hashes; stale values return 409
`POST /api/jobs/{id}/runs`	Approved revision plus idempotency key; queues both routes, returns 202
`GET /api/runs/{id}/events?after=sequence`	Bounded incremental events and current summary
`POST /api/runs/{id}/cancel`	Idempotent cancellation request
`GET /api/attempts/{id}/evidence`	Obligations, scope, counts, changed regions, artifact IDs
`GET /api/artifacts/{id}/download`	Registered regular file only; attachment disposition and safe filename
`POST /api/runs/{id}/exports`	Prepare deterministic report bundle; returns artifact ID

Status codes: 422 invalid structured input, 413 size limit, 401/403 session or origin failure, 404 unavailable object, 409 stale approval or invalid transition. Every response and error carries a correlation ID, and shell tracebacks or secret-bearing provider bodies never reach the browser.

Data model

Proposed SQLite tables — an implementation contract, not a delivered migration.

Table	Key fields
`jobs`	`id` PK, `original_sha256`, `fixture_id` nullable, `classification`, `created_at`, `phase`, `cancel_requested`
`contracts`	`id` PK, `job_id` FK, `revision`, `body_json`, `sha256`, `approved_at` nullable, `reviewer_label`; `UNIQUE(job_id, revision)`
`runs`	`id` PK, `job_id` FK, `contract_id` FK, `manifest_sha256`, `idempotency_key` UNIQUE, `phase`, `outcome` nullable, `started_at`/`finished_at`
`attempts`	`id` PK, `run_id` FK, `route`, `sequence`, `candidate_sha256` nullable, `phase`, `outcome` nullable, `reason_code`; `UNIQUE(run_id, route, sequence)`
`obligations`	`id` PK, `attempt_id` FK, `kind`, `required`, `result`, `scope_sha256`, `evidence_artifact_id` nullable
`artifacts`	`id` PK, `job_id` FK, `attempt_id` nullable, `relative_path`, `sha256`, `size_bytes`, `media_type`, `role`

Table	Key fields
`events`	integer `sequence` PK, `run_id` FK, `attempt_id` nullable, `timestamp`, `event_type`, `payload_json`
`model_calls`	`id` PK, `attempt_id` FK, `provider/model`, `prompt_sha256`, `status`, `usage_json`, `estimated_cost_usd` nullable

Index `events` by run/sequence, `attempts` by run/route, and `jobs` by creation time. Keep large tool output in files, not SQLite rows. Enable foreign keys and set a busy timeout; consider WAL only after a restart/concurrency test, since WAL still permits only one writer.

Phase and outcome are separate. Phases: `draft`, `screening`, `awaiting_review`, `queued`, `analyzing`, `generating`, `applying`, `validating`, `roundtrip` (Route B), `finished`. Write artifacts to a private temp file, hash and validate, atomically rename into the job directory, then register metadata transactionally; reconcile interrupted writes on startup so unregistered files can never satisfy an obligation. SHA-256 establishes identity, not correctness.

Approval binding: an approval binds the original hash, contract revision, test-manifest hash, patch policy, environment profile, and verification scope. Editing any bound input invalidates the approval and requires a new run; earlier runs are preserved.

AI Runtime

- **Provider/runtime:** DeepSeek through a replaceable provider adapter, `request_candidate(context, route, limits) -> validated candidate | typed failure`. Start by evaluating the documented `deepseek-flash` option; pin the exact model/version and settings after a public-case smoke test rather than inferring quality from the model name.
- **Model can see:**
  - Public: public-fixture binary-derived excerpts, recovered C, the contract, relevant diagnostic results.
  - User-owned: nothing beyond the current job's approved evidence, and only the minimum needed rather than all job artifacts.
  - Never send: client binaries, confidential derived content, credentials, API keys, session tokens, private logs, the evaluator's reference fix, or evaluator annotations.
- **Output contract:** JSON output mode plus strict local Pydantic validation. Candidate schema v1 carries `schema_version`, `original_sha256`, `contract_sha256`, `analysis_sha256`, `route`, `target_entry`, `evidence_ids`, Route A `template_id` + typed parameters, Route B `recovered_function_sha256` + `replacement_c`, `expected_effects`, `assumptions`, `uncertainties`. Extra fields are forbidden; hashes, integer ranges, target membership, string sizes, and route-specific fields are validated locally. Valid JSON is not schema validity.
- **The server supplies allowed locations and interfaces.** A candidate cannot expand them, and payloads may not contain host commands, permission requests, credentials, or claimed proof results.
- **Tools/actions:** propose a typed candidate only — no execution, no application, no network, no filesystem access from the model.

- **Approval gates:** reviewer confirmation of location and contract before automated runs; explicit client authorization before any external transfer of client material.
- **Retention/training setting to verify:** provider/account terms, retention and training treatment, region, and any gateway forwarding. **Unverified.** The public privacy policy is not evidence of an API-specific zero-retention agreement. Confirmed public-fixture use may proceed; nothing about confidential-client readiness is implied.
- **Fallback:** on schema failure, empty/truncated content, or exhausted retries, preserve diagnostics and stop or try another candidate within the same caps. Report `error` or `inconclusive`. Never fabricate a response or a positive verification.

Operational defaults (proposed, configurable, not measured)

Limit	Starting value
Active work	One job; sequential route execution
Upload size	20 MiB ELF; 10 MiB combined tests
Candidate generation	At most 3 attempts per route
HTTP model calls	At most 6 per route including retries; 1 transient retry per request with bounded backoff
Model context/output	24,000 input and 4,000 output tokens per call
Provider timeout	120 s per call; record failure and usage where available
Analysis / round-trip step	300 s before explicit timeout
Concrete test	2 s per process, 1 MiB captured output, bounded file writes
Symbolic attempt	120 s and 4 GiB; exceeding either means `inconclusive`
Job	30 minutes and 1 GiB new artifact quota
API spending	USD 2 per job, USD 20 for initial public-case experiments — configurable
Retention	No automatic deletion before milestone review; explicit cleanup after export, review recommended at 30 days

Reserve conservative call cost before sending and bound every billed output category the provider supports. If reliable upper-bound accounting is unavailable, enforce a small call cap and say that monetary control is approximate. Do not silently retry a timed-out billed call. Unknown usage is reported as unavailable, never as zero.

Deliberately excluded for now: vector search, a broad vulnerability database, autonomous multi-agent orchestration, or an agent framework.

Important patterns

- **Data fetching:** server-rendered pages plus small JSON status fetches; JavaScript polls persisted events every 2 s while work runs (SSE and WebSockets were considered and rejected as unnecessary here).

- **State management:** phase and outcome are separate; state lives in SQLite and the artifact store, never only in the browser. Never render a fabricated progress percentage.
- **Forms/validation:** Pydantic models for every request; typed test manifests with argument arrays and artifact IDs that cannot express arbitrary host commands or unrestricted paths.
- **Error handling:** typed failure outcomes with a reason code; a correlation ID on every response; worker errors surface as data, not as a repaired result.
- **Logging/monitoring:** structured local events; stage durations, error reasons, state counts, changed/added bytes, regression counts, usage/cost availability, disk growth, and manual interventions. Exclude keys and confidential payloads. Store large tool output as files.
- **Backup:** snapshot SQLite consistently together with its referenced artifacts, and restore-test before relying on a backup.